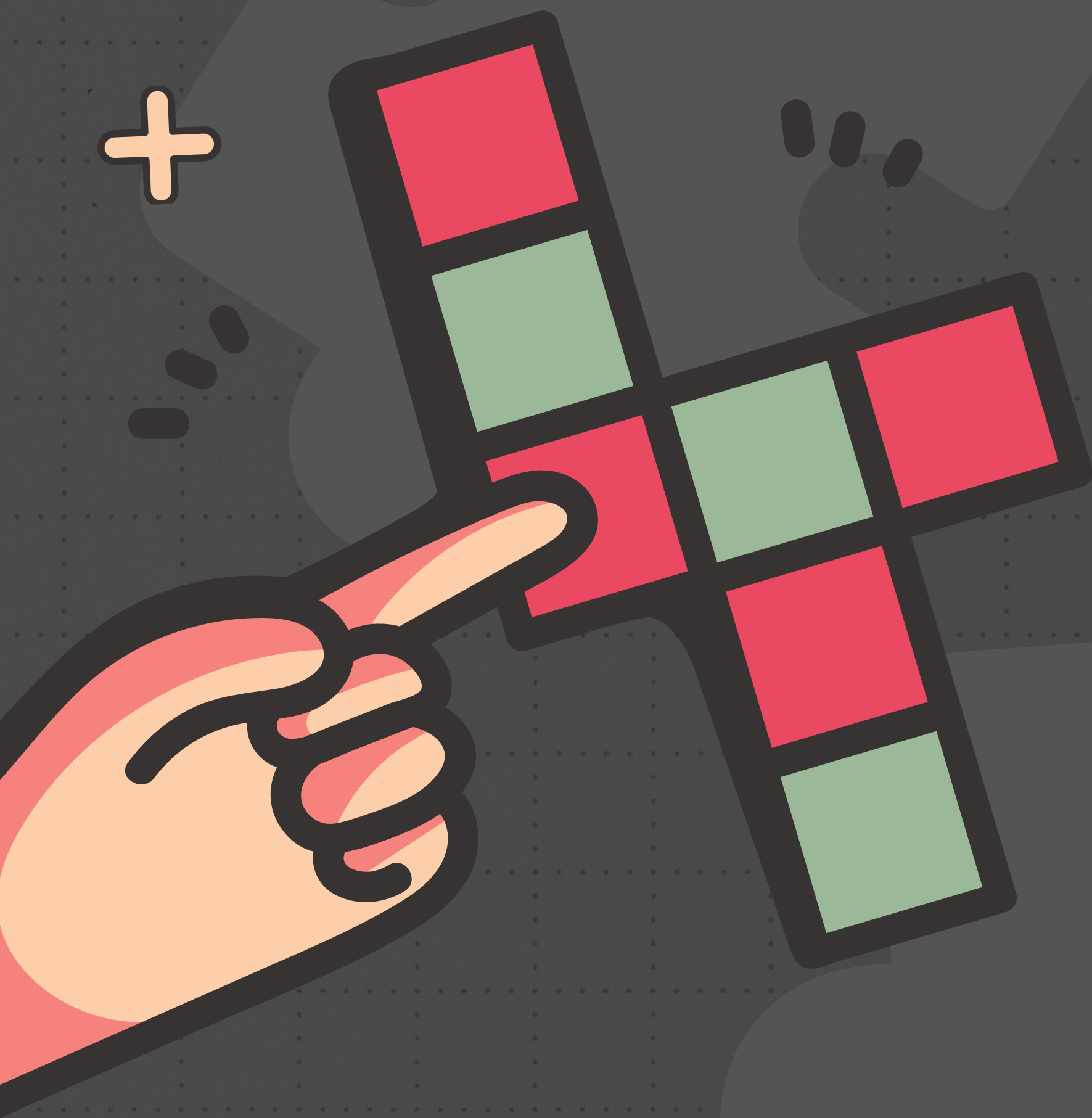




PROJETO DE EDI/

PALAVRAS CRUZADAS COM BACKTRACKING



DOCENTE: RAPHAEL OLIVEIRA
DISCENTES: DAIANE, GUILHERME, IGOR
DIAS, MATHEUS E YASMIM

O QUE É O PROJETO

E COMO JOGAR

O QUE É

Jogo de preencher uma grade com palavras usando pistas.

COMO JOGAR

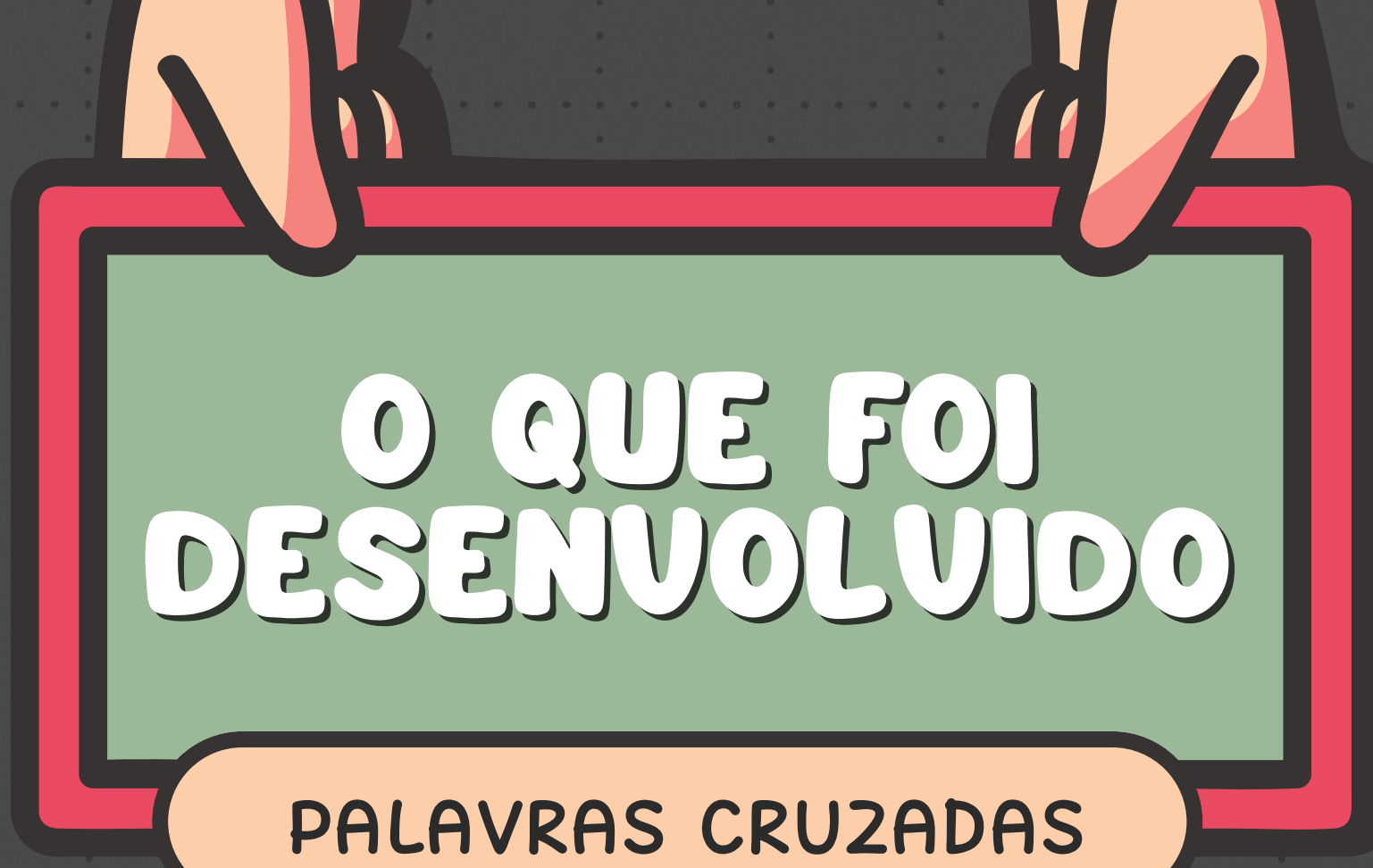
Use as pistas para descobrir palavras e encaixá-las na grade, cruzando letras corretamente.

OBJETIVO DO PROJETO

O objetivo não é apenas jogar, mas demonstrar a aplicação de algoritmos de **Backtracking**, para resolver ou validar o preenchimento do tabuleiro.

TECNOLOGIAS IMPLEMENTADAS

Um jogo de Palavras Cruzadas desenvolvido em C puro, utilizando a biblioteca gráfica Raylib.



O QUE FOI DESENVOLVIDO

PALAVRAS CRUZADAS



INTERFACE GRÁFICA (GUI)

Um tabuleiro interativo estilo "jornal" (azul/branco) onde o usuário pode clicar e digitar.

SISTEMA DE NÍVEIS

Use as pistas para descobrir palavras e encaixá-las na grade, cruzando letras corretamente.

AJUDA INTERATIVA

Dicas (Horizontais/Verticais) e validação de respostas.

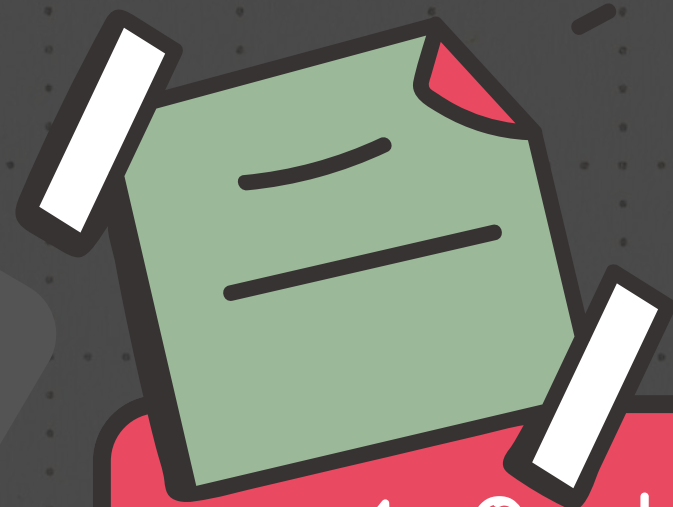
NAVEGAÇÃO INTELIGENTE

Setas do teclado pulam automaticamente os espaços vazios (blocos pretos). Navegação no puzzle por meio de teclado e mouse.



DESTAQUES DO CÓDIGO

PALAVRAS CRUZADAS



ALGORÍTMO DE BACKTRACKING (`SRC/GAME/SOLVER.C`)

1. O algoritmo pega a primeira palavra vazia do grid.
2. Busca no Dicionário todas as palavras que cabem naquele espaço (ex: 5 letras).
3. Tenta colocar uma palavra ("Candidata").
4. **Verifica Conflitos:** A letra cruza corretamente com as palavras que já estão lá?
5. **Recursão:** Se deu certo, chama a si mesmo para a próxima palavra.
6. **Backtrack:** Se chegar num beco sem saída (nenhuma palavra cabe), ele desfaz a última escolha (apaga a palavra) e tenta a próxima candidata.



ALGORÍTMO DE BACKTRACKING (`SRC/GAME/SOLVER.C`)

```
static bool backtrack(Grid *grid, int indicePalavra, EstadoSolver *estado) {
    estado->profundidade = indicePalavra;
    // CASO BASE: Todas as palavras foram preenchidas
    if (indicePalavra >= grid->numPalavras) {
        // Verificar se a solução é válida
        return verificarSolucao(grid);
    }
    // Obter a palavra atual
    Palavra *palavra = obterPalavra(grid, indicePalavra);
    if (palavra == NULL) {
        return false;
    }
    // Se a palavra já está completa, pular para a próxima
    if (palavra->estaCompleta) {
        return backtrack(grid, indicePalavra + 1, estado);
    }
}
```


ALGORÍTMO DE BACKTRACKING (`SRC/GAME/SOLVER.C`)

```
// Obter candidatos do dicionário com o tamanho correto
char **candidatos = NULL;
int numCandidatos = 0;
obterCandidatos(palavra->tamanho, &candidatos, &numCandidatos);
// Tentar cada candidato
for (int i = 0; i < numCandidatos; i++) {
    char *candidato = candidatos[i];
    estado->tentativas++;
    strncpy(estado->ultimaPalavra, candidato, TAMANHO_MAX_PALAVRA);
    estado->ultimaPalavra[TAMANHO_MAX_PALAVRA] = '\0';
    // VALIDAR: Verificar se o candidato pode ser colocado
    if (!validarPalavra(grid, palavra, candidato)) {
        continue; // Candidato inválido, tentar próximo
    }
}
```

ALGORÍTMO DE BACKTRACKING (`SRC/GAME/SOLVER.C`)

```
// COLOCAR: Tentar colocar a palavra no grid
if (!colocarPalavra(grid, palavra, candidato)) {
    continue; // Não conseguiu colocar, tentar próximo
}

// RECURSÃO: Tentar resolver o resto do puzzle
if (backtrack(grid, indicePalavra + 1, estado)) {
    // SUCESSO! Encontrou solução
    liberarCandidatos(candidatos, numCandidatos);
    return true;
}

removerPalavra(grid, palavra); estado->retrocessos++;
}

liberarCandidatos(candidatos, numCandidatos);
return false;
}
```


ALGORÍTMO DE BACKTRACKING (`SRC/GAME/SOLVER.C`)

```
bool resolverPuzzle(Grid *grid, EstadoSolver *estado) {  
    if (grid == NULL || estado == NULL) {  
        return false;  
    }  
    // Inicializar estado  
    inicializarSolver(grid, estado);  
    // Executar backtracking a partir da primeira palavra  
    bool sucesso = backtrack(grid, 0, estado);  
    // Marcar como concluído  
    estado->concluido = true;  
    estado->encontrouSolucao = sucesso;  
    return sucesso;  
}
```



INTEGRAÇÃO COM DICIONÁRIO (`SRC/DATA/DICTIONARY.C`)

- 
- Simula uma API de palavras.
 - Carrega bancos de dados de texto (`pt.txt`, `en.txt`) na memória para o caso de API falhar ou não ter Internet.
 - Permite buscas rápidas por tamanho dentro do algoritmo para montar o puzzle.
- 

INTEGRAÇÃO COM DICIONÁRIO (`SRC/DATA/DICTIONARY.C`)

```
bool dict_check_word(const char* word, char* out_def, int max_len) {  
    for (int i = 0; i < dictWordCount; i++) {  
        if (strcasecmp(dictionary[i].word, word) == 0) {  
            if (strcmp(dictionary[i].lang, currentLang) == 0) {  
                if (out_def) {  
                    strncpy(out_def, dictionary[i].definition, max_len - 1);  
                    out_def[max_len - 1] = '\0';  
                }  
                return true; // Retorna true se encontra no idioma correto  
            }  
        }  
    }  
    return false;  
}
```




BIBLIOTECA GRÁFICA ('RAYLIB') E INTERFACE

- 
- Utilizamos Raylib por ser leve e direta para C.
 - Loop do Jogo: O `'main.c'` roda 60 vezes por segundo, desenhando o grid (`'DrawCrosswordGrid'`) e escutando cliques/teclado (`'UpdateInterface'`).
 - Renderização: Desenhamos retângulo por retângulo, calculando coordenadas X/Y baseadas na matriz do tabuleiro.
- 

BIBLIOTECA GRÁFICA ('RAYLIB') E INTERFACE

```
UpdateInterface(g, &estado);
```

```
// Cabeçalho
```

```
DrawText("Palavras Cruzadas", 40, 20, 30, UI_COLOR_PRIMARY);
```

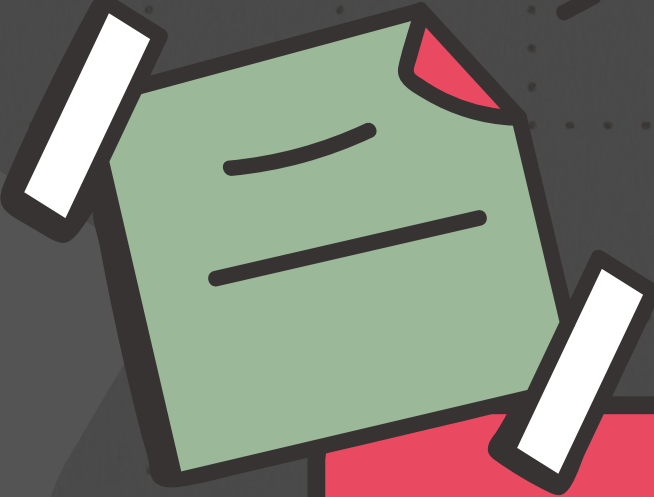
```
char infoStr[64];
```

```
snprintf(infoStr, 64, "Idioma: %s - Nivel %d | Acertos: %d/%d",  
         idiomaAtual, currentLevel, palavrasCorretas, g->numPalavras);
```

```
DrawText(infoStr, 40, 55, 20, DARKGRAY);
```

```
// Grid
```

```
DrawCrosswordGrid(g, 50, 100, 32);
```



CONCLUSÃO



Tivemos desafios como, principalmente na implementação do algoritmo de **backtracking** para a geração procedural infinita de níveis. Garantir que as palavras se cruzassem validamente sem repetições exigiu um refinamento iterativo da lógica recursiva e uma gestão cuidadosa de memória e ponteiros para evitar falhas durante a execução prolongada.

Como aprendizado, destaca-se a importância de separar a lógica de jogo da interface gráfica e o domínio sobre estruturas de dados para otimizar a busca de palavras.





OBRIGADO!

